

R Code Examples from WDDA Lecture 03

WDDA – Data Management and Data Analysis

Prof. Dr. Ulrich Matter

2026-04-29

Contents

1	Introduction	1
2	Preparation	2
3	Measures of Location	2
3.1	Arithmetic Mean	3
3.2	Median	3
3.3	Mode	4
3.4	Weighted Mean	6
4	Measures of Dispersion	7
4.1	Range	7
4.2	Quartiles and Interquartile Range (IQR)	9
4.3	Variance and Standard Deviation	11
5	Empirical Rule	13
6	Standardisation	15
7	Skewness	18
8	Measures of Association	20
8.1	Covariance	20
8.2	Correlation Coefficient	22
9	Visualisation	26
9.1	Boxplots	26

1 Introduction

This document collects all R code examples from the WDDA Lecture 03 slides. The focus is on descriptive statistics, in particular numerical measures such as measures of location, dispersion, and association.

Descriptive statistics is a fundamental area of statistics that deals with summarising, describing, and presenting data. In contrast to inferential statistics, which draws conclusions about a population based on a sample, descriptive statistics describes the data at hand without making any inferences beyond it.

Numerical measures in descriptive statistics can be divided into three main categories:

1. **Measures of location:** describe the centre or central tendency of the data (e.g. mean, median)

2. **Measures of dispersion:** describe the variability or spread of the data (e.g. variance, standard deviation)
3. **Measures of association:** describe the relationship between two or more variables (e.g. covariance, correlation)

2 Preparation

```
# Load packages
library(readxl) # Package for importing Excel files

# Import data
Graduates <- read_excel("../data/WDDA_03.xlsx", sheet = "Graduates")
Brand <- read_excel("../data/WDDA_03.xlsx", sheet = "Auto")
Advertising <- read_excel("../data/WDDA_03.xlsx", sheet = "Advertising")

# Extract variables for later use
Salary <- Graduates$Salary # Extract the column 'Salary' from the Graduates data frame
Major <- Graduates$Major   # Extract the column 'Major' from the Graduates data frame
sales <- Advertising$sales # Extract the column 'sales' from the Advertising data frame
adverts <- Advertising$adverts # Extract the column 'adverts' from the Advertising data
  ↪ frame

# Inspect the data structure
str(Graduates) # Show the structure of the Graduates data frame

## tibble [60 x 2] (S3: tbl_df/tbl/data.frame)
##  $ Major : chr [1:60] "Accounting" "Accounting" "Accounting" "Accounting" ...
##  $ Salary: num [1:60] 3676 4324 4131 4005 3963 ...

head(Salary) # Show the first values of the Salary variable

## [1] 3676 4324 4131 4005 3963 3317
```

R explanation:

- The function `library()` loads an installed package into the current R session.
- With `read_excel()` from the `readxl` package, Excel files can be imported. The `sheet` parameter specifies which worksheet to use.
- The `$` operator is used to access a column of a data frame.
- The function `str()` shows the structure of an object, including data types and dimensions.
- The function `head()` shows the first rows of a data frame or vector.

Note on data paths: In R Markdown files, relative paths are interpreted from the location of the file, which is why the path is `"../data/WDDA_03.xlsx"`. In ordinary R scripts, the path would be specified relative to the working directory.

3 Measures of Location

Measures of location describe the central tendency of a distribution and indicate where the centre of the data lies. They are important summary statistics for getting a first impression of the location of the data.

3.1 Arithmetic Mean

The arithmetic mean is the sum of all data values divided by the number of observations. It is the most commonly used measure of location, but has the disadvantage of being sensitive to outliers.

Mathematical formula: $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$

```
# Manual calculation
sum_Salary <- Salary |> sum() # Calculate the sum of all salaries
n_Salary <- Salary |> length() # Count the number of observations
mean_manual <- sum_Salary / n_Salary # Divide the sum by the count

# Built-in function
mean_builtin <- Salary |> mean() # Use the built-in mean() function

# Output the results
print(paste("Mean (manual):", round(mean_manual, 2)))

## [1] "Mean (manual): 3630.72"

print(paste("Mean (built-in):", round(mean_builtin, 2)))

## [1] "Mean (built-in): 3630.72"

# Demonstration of sensitivity to outliers
salary_with_outlier <- c(Salary, 1000000) # Add an extreme outlier
print(paste("Mean without outlier:", round(mean(Salary), 2)))

## [1] "Mean without outlier: 3630.72"

print(paste("Mean with outlier:", round(mean(salary_with_outlier), 2)))

## [1] "Mean with outlier: 19964.64"
```

Statistical explanation: The arithmetic mean is a centre of mass of the data. It minimises the sum of squared deviations and is therefore optimal for symmetric distributions without outliers. For skewed distributions or in the presence of outliers, however, the mean can give a distorted picture of the central tendency.

R explanation:

- The function `sum()` calculates the sum of all values in a vector.
- The function `length()` returns the number of elements in a vector.
- The function `mean()` directly calculates the arithmetic mean.
- The pipe operator `|>` passes the result of the left-hand expression as the first argument to the right-hand function.
- The function `round()` rounds to a specified number of decimal places.

3.2 Median

The median splits the ordered data into two halves. It is robust to outliers and therefore particularly useful for skewed distributions or data with extreme values.

Mathematical definition:

- For odd n : the value at position $(n+1)/2$ in the ordered series

- For even n : the mean of the values at positions $n/2$ and $n/2+1$

```
# Calculation of the median using the built-in function
median_salary <- Salary |> median()
print(paste("Median:", median_salary))

## [1] "Median: 3663"

# Manual calculation for demonstration
sorted_salary <- sort(Salary) # Sort the salaries in ascending order
n <- length(sorted_salary)    # Number of observations

# Determine the median depending on whether the count is even or odd
if (n %% 2 == 1) { # Odd count
  median_manual <- sorted_salary[(n+1)/2]
} else {          # Even count
  median_manual <- (sorted_salary[n/2] + sorted_salary[n/2 + 1]) / 2
}
print(paste("Median (manual):", median_manual))

## [1] "Median (manual): 3663"

# Comparison of median and mean in the presence of outliers
print(paste("Median with outlier:", median(salary_with_outlier)))

## [1] "Median with outlier: 3676"

print(paste("Mean with outlier:", round(mean(salary_with_outlier), 2)))

## [1] "Mean with outlier: 19964.64"
```

Statistical explanation: The median is a robust measure of location that splits the data into two equally sized halves. In contrast to the mean, it is barely affected by outliers, since it depends only on the rank order of the data, not on their exact values. The median is particularly suitable for:

- Skewed distributions (e.g. income distributions)
- Data with outliers
- Ordinal-scale data (where only the rank order, but not the exact distances, are known)

R explanation:

- The function `median()` calculates the median of a vector.
- The function `sort()` sorts a vector in ascending order.
- The modulo operator `%%` returns the remainder of a division and is used here to check whether the count is even or odd.

3.3 Mode

The mode is the most frequently occurring value in a dataset. It is particularly useful for categorical data and, unlike the mean and median, can also be calculated for nominal-scale data.

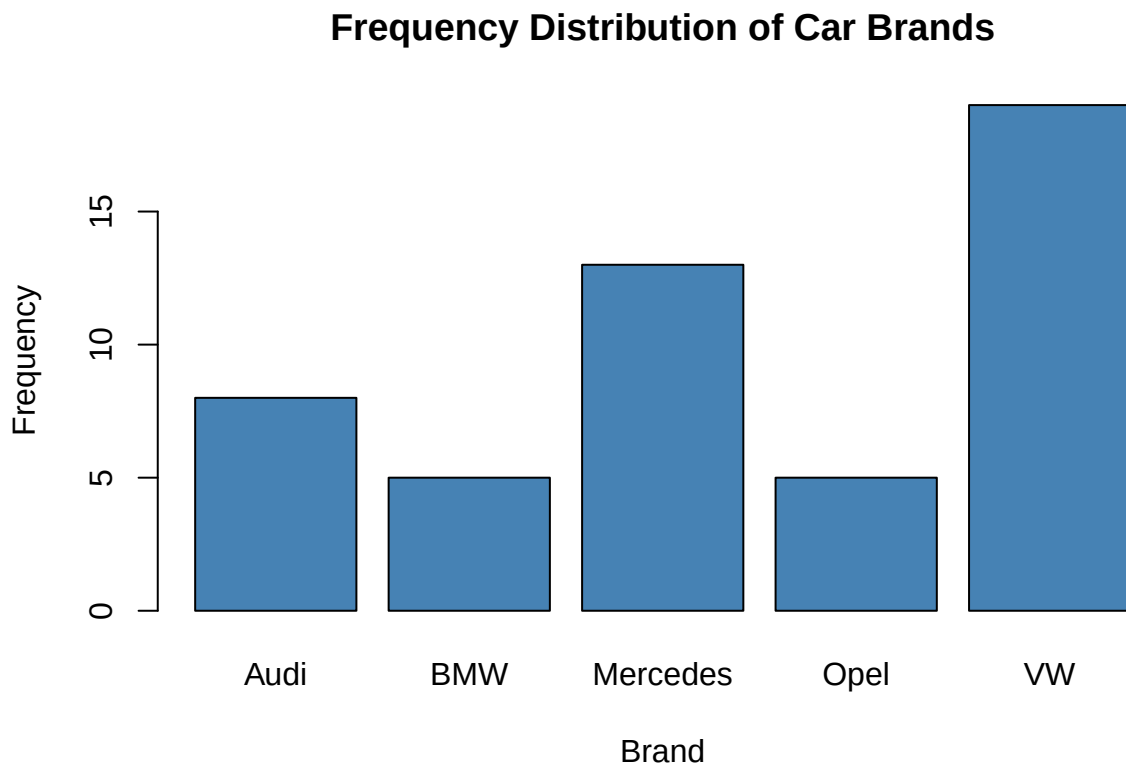
```
# Create a frequency table
brand_table <- Brand |> table()
print(brand_table)
```

```
## Brand
##      Audi      BMW Mercedes      Opel      VW
##        8        5        13        5        19

# Determine the mode
mode_brand <- brand_table |>
  which.max() |> # Returns the position of the largest value
  names()        # Extracts the name (the category) at this position
print(paste("Mode:", mode_brand))

## [1] "Mode: VW"
```

```
# Visualisation of the frequency distribution
bp <- barplot(brand_table,
  main = "Frequency Distribution of Car Brands",
  xlab = "Brand",
  ylab = "Frequency",
  col = "steelblue")
# Mark the mode
text(bp[which(names(brand_table) == mode_brand)],
  brand_table[mode_brand] + 1,
  "Mode",
  col = "red")
```



Statistical explanation: The mode is the only measure of location that is applicable to all measurement scales (nominal, ordinal, interval, ratio). A dataset can be unimodal (one mode), bimodal (two modes), or multimodal (several modes). For continuous data, the mode is often defined as the value at which the probability density function reaches its maximum.

R explanation:

- The function `table()` creates a frequency table that counts the number of occurrences for each unique category.
- The function `which.max()` returns the index (position) of the largest value in a vector.
- The function `names()` extracts the names of a named vector or table.
- R has no built-in function for the mode, so it is typically calculated using a combination of `table()` and `which.max()`.

Note: In the case of ties (several values with the same highest frequency), `which.max()` returns only the first one. To find all modes, one would have to identify all positions where the frequency equals the maximum.

3.4 Weighted Mean

In the weighted mean, each data value is assigned a weight that reflects its relative importance. This is particularly useful when not all observations are equally important or when data is already given in aggregated form.

Mathematical formula: $\bar{x}_w = \frac{\sum_{i=1}^n w_i x_i}{\sum_{i=1}^n w_i}$

```
# Example: calculation of the weighted mean
xvar <- c(3, 3.4, 2.8, 2.9, 3.25) # Values
weight <- c(1200, 500, 2750, 1000, 800) # Weights

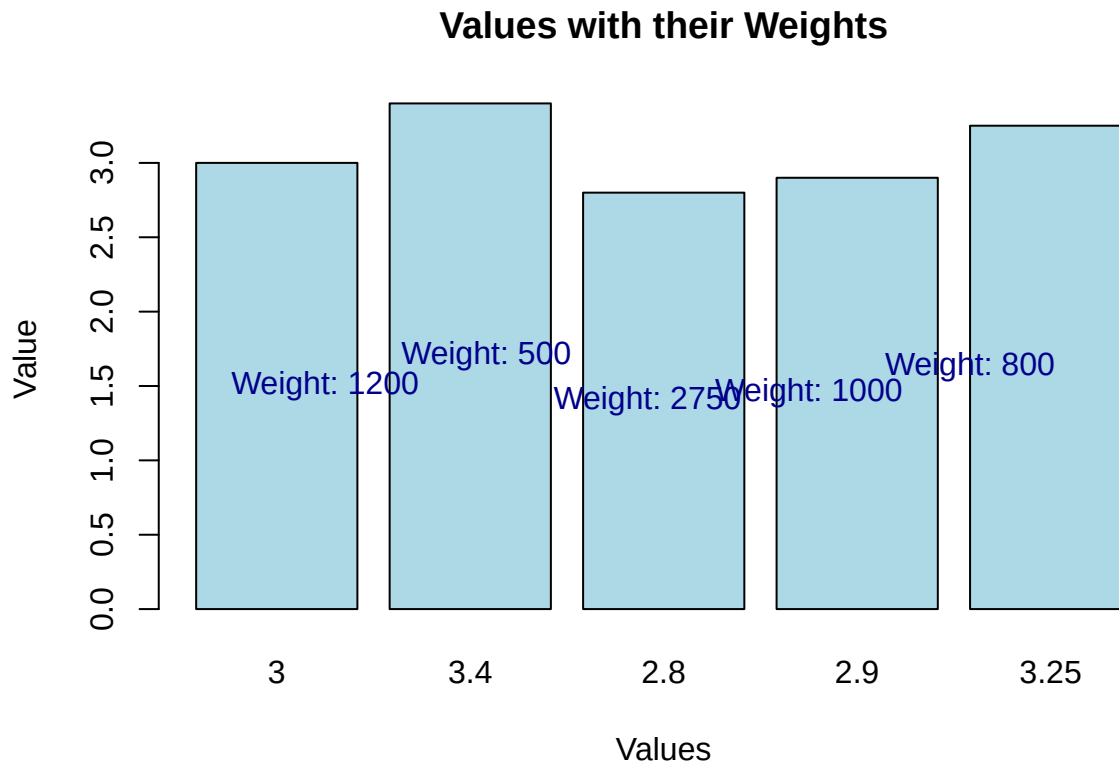
# Calculation of the weighted mean
weighted_mean <- sum(xvar * weight) / sum(weight)
print(paste("Weighted mean:", round(weighted_mean, 4)))
```

```
## [1] "Weighted mean: 2.96"
```

```
# Comparison with the unweighted mean
unweighted_mean <- mean(xvar)
print(paste("Unweighted mean:", round(unweighted_mean, 4)))
```

```
## [1] "Unweighted mean: 3.07"
```

```
# Visualisation of the weighting
barplot(xvar,
        names.arg = xvar,
        col = "lightblue",
        main = "Values with their Weights",
        xlab = "Values",
        ylab = "Value")
# Add weights as text
text(1:length(xvar), xvar/2,
     labels = paste("Weight:", weight),
     col = "darkblue")
```



Statistical explanation: The weighted mean accounts for the differing importance or frequency of individual observations. Typical use cases are:

1. **Aggregated data:** when data is already grouped (e.g. average salaries by department with different numbers of employees)
2. **Unequal sample sizes:** when different subsamples are of different size
3. **Differences in quality:** when some measurements are more reliable than others
4. **Time series:** when calculating moving averages, where more recent data is often weighted more heavily

R explanation:

- Element-wise multiplication of vectors in R is performed with the `*` operator.
- The function `weighted.mean(x, w)` is a built-in alternative to the manual calculation.
- When visualising with `barplot()`, the parameter `names.arg` can be used to label the bars.

4 Measures of Dispersion

Measures of dispersion describe how strongly the data are spread around the centre (location measure). They provide information about the variability or spread of the data and are important for assessing the homogeneity or heterogeneity of a dataset.

4.1 Range

The range is the difference between the largest and the smallest value in a dataset. It is the simplest measure of dispersion, but has the disadvantage that it depends on only two extreme values and says nothing about the distribution between them.

```
max_salary <- Salary |> max() # Largest value
min_salary <- Salary |> min() # Smallest value
```

```
range_salary <- max_salary - min_salary # Difference
print(paste("Maximum:", max_salary))

## [1] "Maximum: 4568"

print(paste("Minimum:", min_salary))

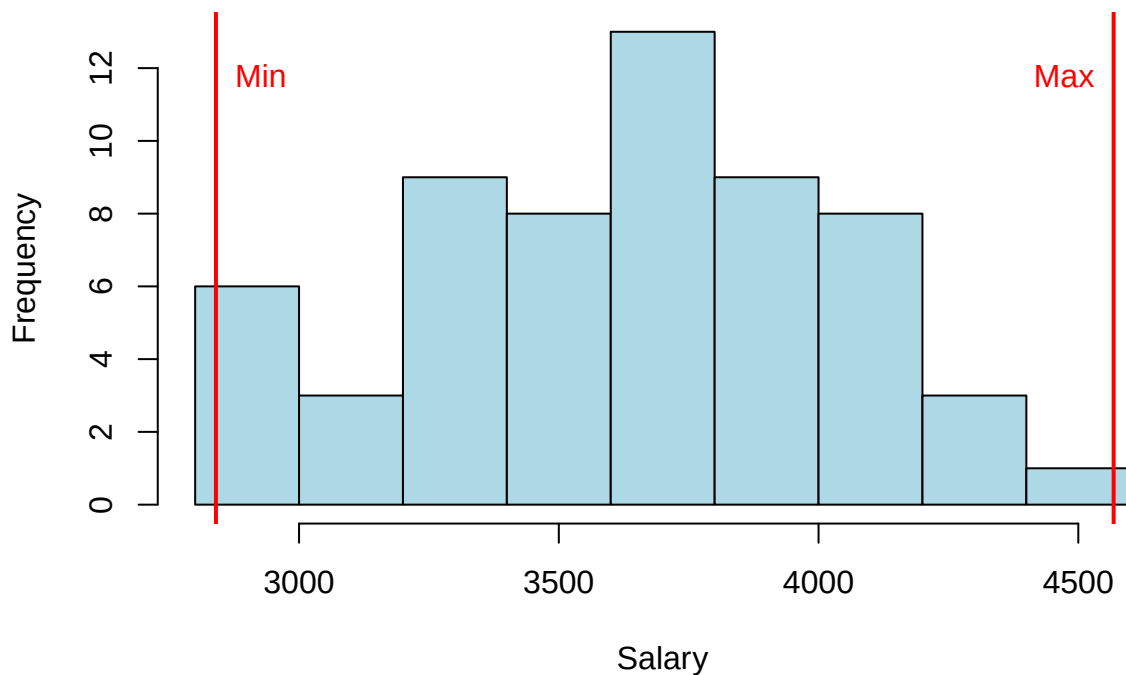
## [1] "Minimum: 2840"

print(paste("Range:", range_salary))

## [1] "Range: 1728"

# Visualisation of the range
hist(Salary,
     main = "Histogram of Salaries with Range",
     xlab = "Salary",
     col = "lightblue")
abline(v = min_salary, col = "red", lwd = 2)
abline(v = max_salary, col = "red", lwd = 2)
text(min_salary, max(hist(Salary, plot = FALSE)$counts) * 0.9,
     "Min", pos = 4, col = "red")
text(max_salary, max(hist(Salary, plot = FALSE)$counts) * 0.9,
     "Max", pos = 2, col = "red")
```

Histogram of Salaries with Range



Statistical explanation: The range gives a first impression of the spread of the data, but it is very sensitive to outliers. It is often reported as supplementary information alongside more robust measures of dispersion

such as the interquartile range. The range is particularly useful for:

- A quick overview of the overall spread
- Identifying possible outliers
- Setting axis ranges in plots

R explanation:

- The functions `max()` and `min()` return the largest and smallest value of a vector, respectively.
- The function `abline()` with the parameter `v` adds vertical lines to an existing plot.
- The function `hist()` with `plot = FALSE` computes the histogram statistics without drawing them.

4.2 Quartiles and Interquartile Range (IQR)

Quartiles divide the ordered data into four equal parts. The interquartile range (IQR) is the difference between the third quartile (Q3) and the first quartile (Q1), and covers the middle 50% of the data. It is a robust measure of dispersion that is not affected by outliers.

```
# Calculation of the quartiles
Q1 <- Salary |> quantile(0.25) # First quartile (25th percentile)
Q2 <- Salary |> quantile(0.50) # Second quartile = median (50th percentile)
Q3 <- Salary |> quantile(0.75) # Third quartile (75th percentile)

# Calculation of the interquartile range
IQR_salary <- Q3 - Q1

print(paste("Q1 (25%):", Q1))
```

```
## [1] "Q1 (25%): 3337.75"
```

```
print(paste("Q2 (median, 50%):", Q2))
```

```
## [1] "Q2 (median, 50%): 3663"
```

```
print(paste("Q3 (75%):", Q3))
```

```
## [1] "Q3 (75%): 3969"
```

```
print(paste("IQR:", IQR_salary))
```

```
## [1] "IQR: 631.25"
```

```
# Percentiles
percentile10 <- Salary |> quantile(0.10) # 10th percentile
percentile95 <- Salary |> quantile(0.95) # 95th percentile
print(paste("10th percentile:", percentile10))
```

```
## [1] "10th percentile: 3046.2"
```

```
print(paste("95th percentile:", percentile95))
```

```
## [1] "95th percentile: 4241.35"
```

```

# Identification of outliers using the 1.5*IQR rule
lower_fence <- Q1 - 1.5 * IQR_salary
upper_fence <- Q3 + 1.5 * IQR_salary
print(paste("Lower outlier fence:", lower_fence))

## [1] "Lower outlier fence: 2390.875"

print(paste("Upper outlier fence:", upper_fence))

## [1] "Upper outlier fence: 4915.875"

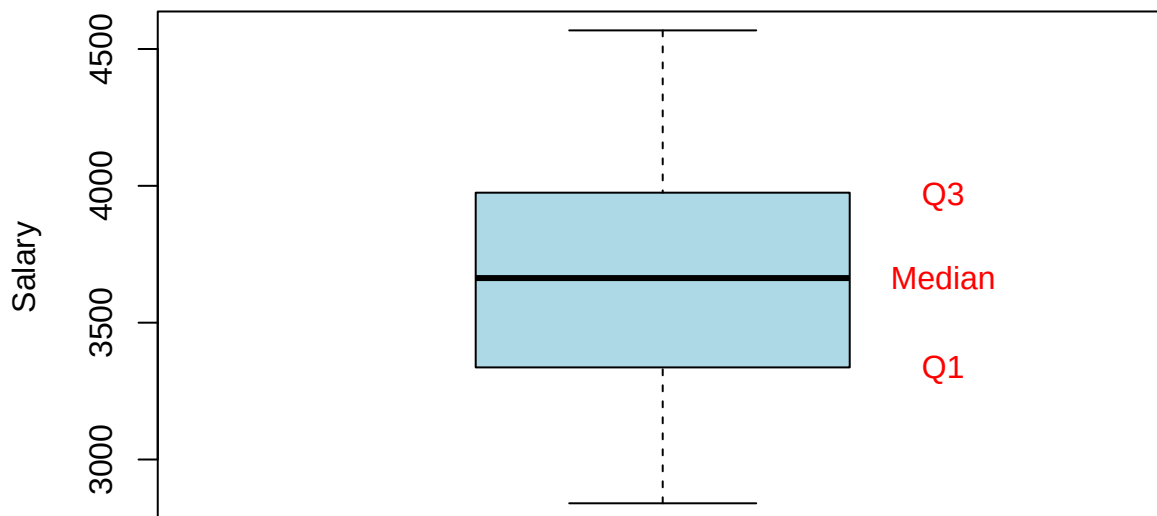
# Number of outliers
outliers <- Salary[Salary < lower_fence | Salary > upper_fence]
print(paste("Number of outliers:", length(outliers)))

## [1] "Number of outliers: 0"

# Visualisation with a boxplot
boxplot(Salary,
        main = "Boxplot of Salaries with Quartiles",
        ylab = "Salary",
        col = "lightblue")
text(1.3, Q1, "Q1", col = "red")
text(1.3, Q2, "Median", col = "red")
text(1.3, Q3, "Q3", col = "red")
text(1.3, upper_fence, "Upper outlier fence", col = "darkred")
text(1.3, lower_fence, "Lower outlier fence", col = "darkred")

```

Boxplot of Salaries with Quartiles



Statistical explanation:

- **Quartiles** divide the ordered data into four equally sized groups:
 - Q1 (25th percentile): 25% of the data lies below it

- Q2 (50th percentile = median): 50% of the data lies below it
- Q3 (75th percentile): 75% of the data lies below it
- The **interquartile range (IQR)** = Q3 - Q1 covers the middle 50% of the data
- The ****1.5*IQR rule**** is commonly used to identify outliers:
 - Values $< Q1 - 1.5IQR$ or $> Q3 + 1.5IQR$ are considered outliers
 - These bounds are shown as the “whiskers” of a boxplot

R explanation:

- The function `quantile()` calculates quantiles (percentiles) of a vector. The second argument specifies the percentage as a decimal.
- The function `IQR()` is a built-in alternative to the manual calculation of the interquartile range.
- In boxplots, outliers are typically displayed as individual points.

4.3 Variance and Standard Deviation

The variance measures the average squared deviation from the mean and is a central measure of the spread of the data. The standard deviation is the square root of the variance and has the advantage of being in the same unit as the original data.

Mathematical formulas:

- Variance (sample): $s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$
- Standard deviation (sample): $s = \sqrt{s^2}$

```
# Variance
var_manual <- Salary |>
  (\(x) sum((x - mean(x))^2))() / (length(Salary) - 1) # Anonymous function with R pipe
var_builtin <- Salary |> var() # Built-in function

# Standard deviation
sd_manual <- Salary |> var() |> sqrt() # Square root of the variance
sd_builtin <- Salary |> sd() # Built-in function

print(paste("Variance (manual):", round(var_manual, 2)))
```

```
## [1] "Variance (manual): 168789.77"
```

```
print(paste("Variance (built-in):", round(var_builtin, 2)))
```

```
## [1] "Variance (built-in): 168789.77"
```

```
print(paste("Standard deviation (manual):", round(sd_manual, 2)))
```

```
## [1] "Standard deviation (manual): 410.84"
```

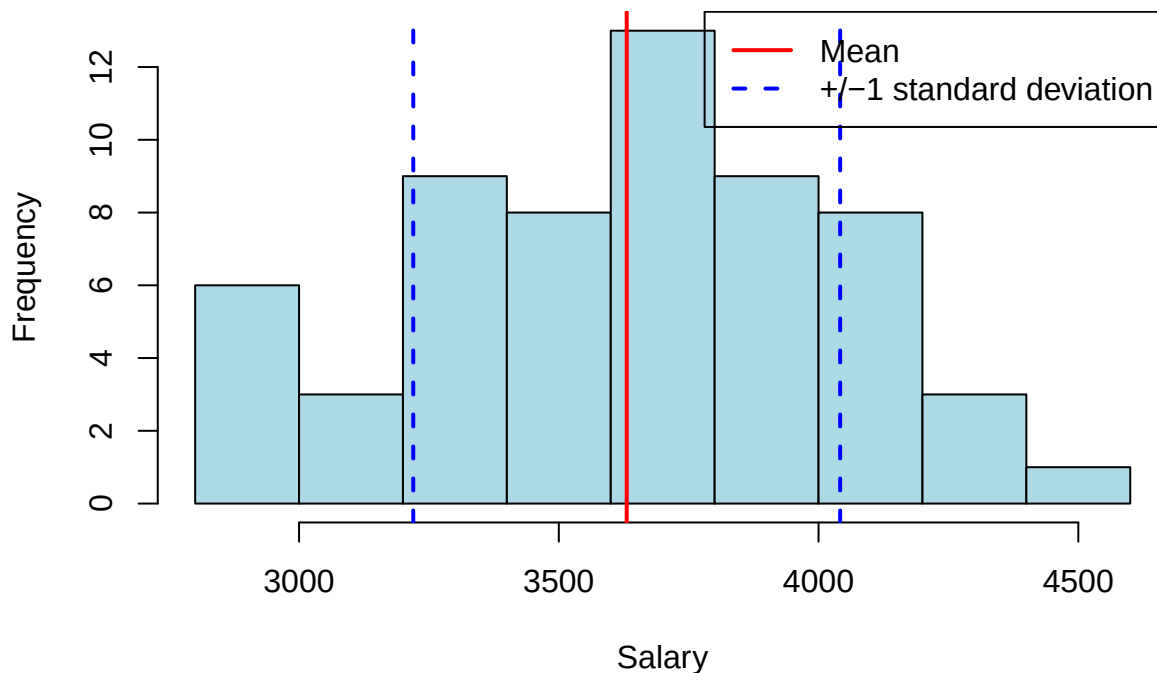
```
print(paste("Standard deviation (built-in):", round(sd_builtin, 2)))
```

```
## [1] "Standard deviation (built-in): 410.84"
```

```
# Visualisation of the standard deviation
hist(Salary,
      main = "Histogram of Salaries with Standard Deviation",
      xlab = "Salary",
```

```
col = "lightblue")
abline(v = mean(Salary), col = "red", lwd = 2)
abline(v = mean(Salary) + sd(Salary), col = "blue", lwd = 2, lty = 2)
abline(v = mean(Salary) - sd(Salary), col = "blue", lwd = 2, lty = 2)
legend("topright",
      legend = c("Mean", "+/-1 standard deviation"),
      col = c("red", "blue"),
      lwd = 2,
      lty = c(1, 2))
```

Histogram of Salaries with Standard Deviation



Statistical explanation:

- The **variance** measures how far the data are from the mean on average. The deviations are squared so that negative and positive deviations are weighted equally.
- When calculating the sample variance, division is by $(n-1)$ instead of n , in order to obtain an unbiased estimate of the population variance (Bessel's correction).
- The **standard deviation** is the square root of the variance and has the advantage of being in the same unit as the original data, which makes interpretation easier.
- For normally distributed data, about 68% of the values lie within one standard deviation of the mean (empirical rule).

R explanation:

- The anonymous function `\(x) ...` is a new syntax in R (since version 4.1.0) for anonymous functions.
- The parentheses `()` after the anonymous function execute it immediately.
- The functions `var()` and `sd()` calculate the sample variance and sample standard deviation, respectively.
- The parameter `lty` in `abline()` controls the line type (1 = solid, 2 = dashed).
- The function `legend()` adds a legend to the plot.

Tip: The standard deviation has the advantage of being in the same unit as the original data, which makes interpretation easier. It is particularly useful for:

- Comparisons between different datasets
- Assessing the precision of measurements
- Calculating confidence intervals

5 Empirical Rule

The empirical rule (also known as the 68-95-99.7 rule) describes the distribution of data around the mean for approximately normally distributed data. It states that:

- About 68% of the data lie within one standard deviation of the mean
- About 95% of the data lie within two standard deviations of the mean
- About 99.7% of the data lie within three standard deviations of the mean

```
# Mean and standard deviation
mean_salary <- mean(Salary)
sd_salary <- sd(Salary)

# Interval [mean - sd, mean + sd] (68% rule)
lower_1sd <- mean_salary - sd_salary
upper_1sd <- mean_salary + sd_salary
print(paste("Interval [mean +/- 1sd]:", round(lower_1sd, 2), "to", round(upper_1sd, 2)))
```

```
## [1] "Interval [mean +/- 1sd]: 3219.88 to 4041.56"
```

```
# Interval [mean - 2sd, mean + 2sd] (95% rule)
lower_2sd <- mean_salary - 2*sd_salary
upper_2sd <- mean_salary + 2*sd_salary
print(paste("Interval [mean +/- 2sd]:", round(lower_2sd, 2), "to", round(upper_2sd, 2)))
```

```
## [1] "Interval [mean +/- 2sd]: 2809.04 to 4452.4"
```

```
# Interval [mean - 3sd, mean + 3sd] (99.7% rule)
lower_3sd <- mean_salary - 3*sd_salary
upper_3sd <- mean_salary + 3*sd_salary
print(paste("Interval [mean +/- 3sd]:", round(lower_3sd, 2), "to", round(upper_3sd, 2)))
```

```
## [1] "Interval [mean +/- 3sd]: 2398.2 to 4863.24"
```

```
# Calculate the proportion of data in these intervals
prop_1sd <- mean(Salary >= lower_1sd & Salary <= upper_1sd)
prop_2sd <- mean(Salary >= lower_2sd & Salary <= upper_2sd)
prop_3sd <- mean(Salary >= lower_3sd & Salary <= upper_3sd)

print(paste("Proportion of data in 1sd interval:", round(prop_1sd * 100, 1), "%
↳ (expected: ~68%)"))
```

```
## [1] "Proportion of data in 1sd interval: 70 % (expected: ~68%)"
```

```
print(paste("Proportion of data in 2sd interval:", round(prop_2sd * 100, 1), "%
↳ (expected: ~95%)"))
```

```
## [1] "Proportion of data in 2sd interval: 98.3 % (expected: ~95%)"
```

```
print(paste("Proportion of data in 3sd interval:", round(prop_3sd * 100, 1), "%  
↪ (expected: ~99.7%)"))
```

```
## [1] "Proportion of data in 3sd interval: 100 % (expected: ~99.7%)"
```

```
# Visualisation of the empirical rule
# Extend the range of x to show the full curve
x_range <- max(Salary) - min(Salary)
x <- seq(min(Salary) - 0.2*x_range, max(Salary) + 0.2*x_range, length.out = 200)
y <- dnorm(x, mean = mean_salary, sd = sd_salary)

# Compute the maximum density for the y-axis scaling
y_max <- max(y) * 1.1 # 10% buffer above the maximum

plot(x, y, type = "l", lwd = 2, col = "blue",
     main = "Normal Distribution with Empirical Rule",
     xlab = "Salary", ylab = "Density",
     xlim = c(min(x), max(x)), # Set explicit x limits
     ylim = c(0, y_max)) # Set explicit y limits

# Fill the regions under the curve
polygon(c(lower_1sd, x[x >= lower_1sd & x <= upper_1sd], upper_1sd),
       c(0, y[x >= lower_1sd & x <= upper_1sd], 0),
       col = rgb(1, 0, 0, 0.3)) # 68% (+/-1sd) in red

polygon(c(lower_2sd, x[x >= lower_2sd & x < lower_1sd], lower_1sd),
       c(0, y[x >= lower_2sd & x < lower_1sd], 0),
       col = rgb(0, 1, 0, 0.3)) # Additional 13.5% on the left (+/-2sd) in green

polygon(c(upper_1sd, x[x > upper_1sd & x <= upper_2sd], upper_2sd),
       c(0, y[x > upper_1sd & x <= upper_2sd], 0),
       col = rgb(0, 1, 0, 0.3)) # Additional 13.5% on the right (+/-2sd) in green

# Legend
legend("topright",
      legend = c("68% (+/-1sd)", "95% (+/-2sd)"),
      fill = c(rgb(1, 0, 0, 0.3), rgb(0, 1, 0, 0.3)))
```

Normal Distribution with Empirical Rule



Statistical explanation: The empirical rule is a rule of thumb for the distribution of data in a normal distribution. It is useful for:

- Quickly estimating which proportion of the data lies in particular ranges
- Identifying potential outliers (values outside three standard deviations)
- Assessing whether data is approximately normally distributed (by comparing the actual proportions with the expected ones)

Deviations of the actual proportions from the expected values (68%, 95%, 99.7%) can give hints about the shape of the distribution:

- Larger proportions in the intervals indicate a more peaked distribution (higher kurtosis)
- Smaller proportions indicate heavier tails (lower kurtosis)

R explanation:

- The function `mean()` applied to a logical vector calculates the proportion of `TRUE` values.
- The function `dnorm()` calculates the density of the normal distribution at given points.
- The function `polygon()` fills a region under a curve.
- The function `rgb()` creates colours with transparency (the fourth parameter is `alpha`).

6 Standardisation

Standardisation (z-transformation) makes different variables comparable by bringing them to a common scale with mean 0 and standard deviation 1. This is particularly useful when variables with different units or orders of magnitude are to be compared.

Mathematical formula: $z = \frac{x - \mu}{\sigma}$, where μ is the mean and σ is the standard deviation.

```
# Standardisation of the salary data
Salary_z <- (Salary - (Salary |> mean())) / (Salary |> sd())
```

```

head(Salary_z)  # Show the first standardised values

## [1]  0.1102213  1.6874764  1.2177075  0.9110190  0.8087895 -0.7635976

# Verify the properties of the standardised data
print(paste("Mean of z-values:", round(mean(Salary_z), 10))) # Should be close to 0

## [1] "Mean of z-values: 0"

print(paste("Standard deviation of z-values:", round(sd(Salary_z), 10))) # Should be
↪ close to 1

## [1] "Standard deviation of z-values: 1"

# Example: standardisation of a second variable for comparison
# Suppose we also have data on work experience in years
Experience <- c(5, 3, 8, 12, 7, 10, 15, 2, 6, 9, 11, 4, 8, 7, 5)

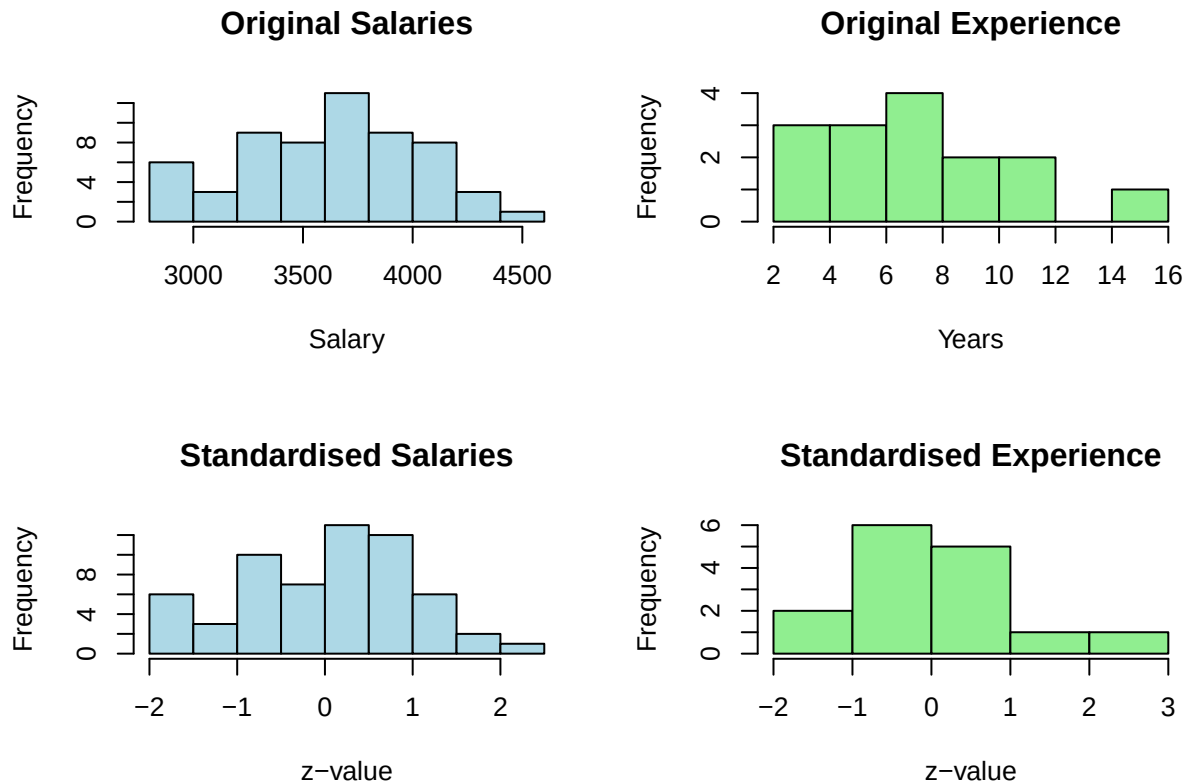
# Standardisation of both variables
Experience_z <- (Experience - mean(Experience)) / sd(Experience)

# Comparison of the original and standardised values
par(mfrow = c(2, 2)) # 2x2 plot layout

# Original values
hist(Salary, main = "Original Salaries", xlab = "Salary", col = "lightblue")
hist(Experience, main = "Original Experience", xlab = "Years", col = "lightgreen")

# Standardised values
hist(Salary_z, main = "Standardised Salaries", xlab = "z-value", col = "lightblue")
hist(Experience_z, main = "Standardised Experience", xlab = "z-value", col =
↪ "lightgreen")

```

```
# Reset to a single plot
par(mfrow = c(1, 1))

# Interpretation of the z-values
# Example: identification of values that are more than two standard deviations from the
# ↪ mean
extreme_values <- Salary[abs(Salary_z) > 2]
print(paste("Number of extreme values (|z| > 2):", length(extreme_values)))

## [1] "Number of extreme values (|z| > 2): 1"

if (length(extreme_values) > 0) {
  print("Extreme values:")
  print(extreme_values)
}

## [1] "Extreme values:"
## [1] 4568
```

Statistical explanation: Standardisation (z-transformation) has several important applications:

1. **Comparability:** variables with different units or orders of magnitude can be compared directly.
2. **Interpretation:** z-values indicate how many standard deviations a value is from the mean.
 - $z = 0$: the value is exactly at the mean
 - $z = 1$: the value lies one standard deviation above the mean
 - $z = -1$: the value lies one standard deviation below the mean
3. **Outlier detection:** values with $|z| > 2$ or $|z| > 3$ are often regarded as potential outliers.
4. **Prerequisite for many statistical methods:** many multivariate methods assume standardised vari-

ables.

R explanation:

- Standardisation can be performed manually with the formula $(x - \text{mean}(x)) / \text{sd}(x)$.
- The function `scale()` is a built-in alternative: `scale(x)` standardises a vector or the columns of a matrix.
- The function `par(mfrow = c(rows, cols))` divides the plotting window into a matrix with `rows` rows and `cols` columns.
- The function `abs()` returns the absolute value of a number.

7 Skewness

Skewness measures the asymmetry of a distribution. It indicates whether the distribution is symmetric or whether it has a longer tail on one side.

Mathematical formula for sample skewness: $g_1 = \frac{n}{(n-1)(n-2)} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{s} \right)^3$

```
# Manual calculation of skewness using standardised values
n <- Salary_z |> length()
skew <- n / ((n - 1) * (n - 2)) * (Salary_z^3 |> sum())
print(paste("Skewness (manual):", round(skew, 4)))
```

```
## [1] "Skewness (manual): -0.1395"
```

```
# Alternative using the e1071 package
if (!requireNamespace("e1071", quietly = TRUE)) {
  install.packages("e1071")
}
library(e1071)
skew_e1071 <- skewness(Salary)
print(paste("Skewness (with e1071):", round(skew_e1071, 4)))
```

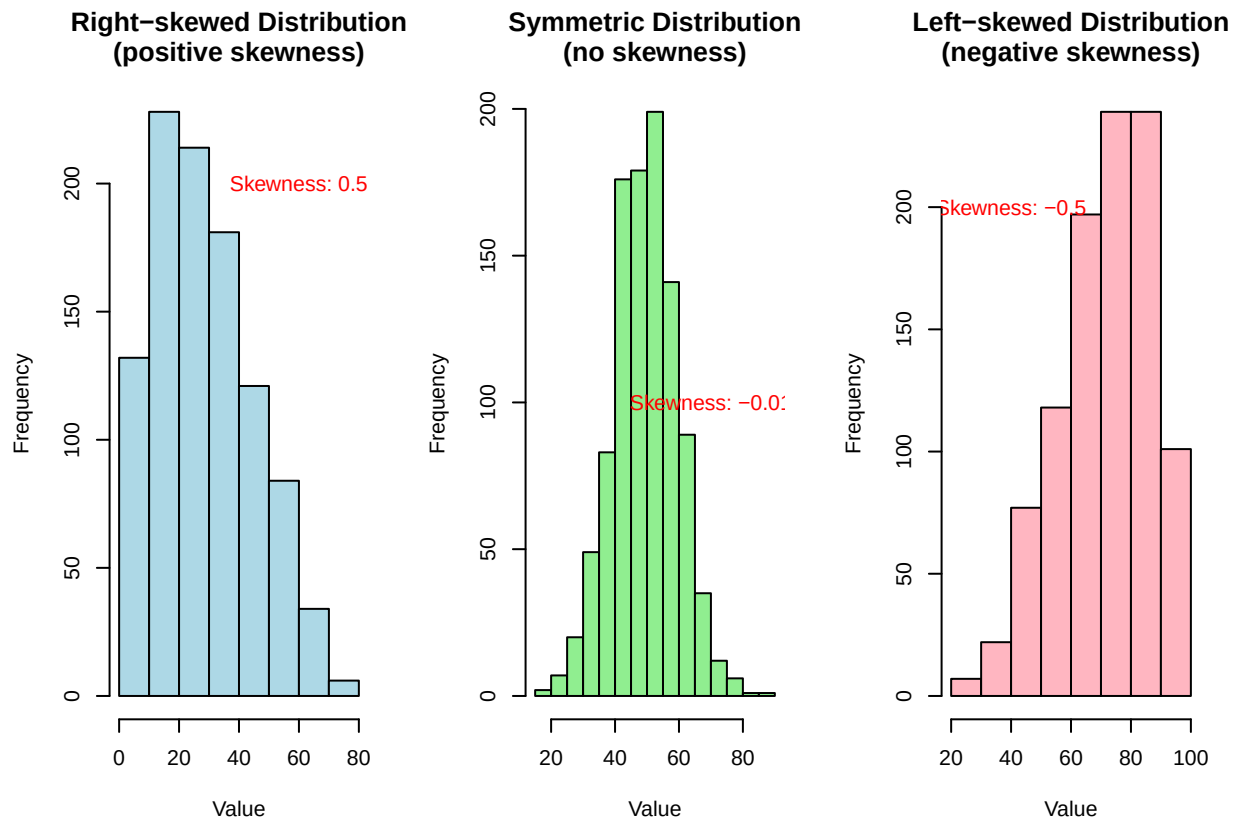
```
## [1] "Skewness (with e1071): -0.1326"
```

```
# Visualisation of different types of skewness
par(mfrow = c(1, 3))

# Right-skewed distribution (positive skewness)
x_right <- rbeta(1000, 2, 5) * 100 # Beta distribution with positive skewness
hist(x_right,
     main = "Right-skewed Distribution\n(positive skewness)",
     xlab = "Value",
     col = "lightblue")
text(60, 200, paste("Skewness:", round(skewness(x_right), 2)), col = "red")

# Symmetric distribution (no skewness)
x_symm <- rnorm(1000, 50, 10) # Normal distribution (symmetric)
hist(x_symm,
     main = "Symmetric Distribution\n(no skewness)",
     xlab = "Value",
     col = "lightgreen")
text(70, 100, paste("Skewness:", round(skewness(x_symm), 2)), col = "red")
```

```
# Left-skewed distribution (negative skewness)
x_left <- 100 - rbeta(1000, 2, 5) * 100 # Reversed beta distribution
hist(x_left,
     main = "Left-skewed Distribution\n(negative skewness)",
     xlab = "Value",
     col = "lightpink")
text(40, 200, paste("Skewness:", round(skewness(x_left), 2)), col = "red")
```



```
# Reset to a single plot
par(mfrow = c(1, 1))

# Interpretation of the skewness of the salary data
if (skew > 0.5) {
  print("The salary data is clearly right-skewed (positive skewness).")
  print("This means that there are a few high salaries that pull the mean upwards.")
  print("In this case, the median is often more informative than the mean.")
} else if (skew < -0.5) {
  print("The salary data is clearly left-skewed (negative skewness).")
  print("This means that there are a few low salaries that pull the mean downwards.")
} else {
  print("The salary data is approximately symmetric.")
  print("Mean and median are close to each other.")
}
```

```
## [1] "The salary data is approximately symmetric."
## [1] "Mean and median are close to each other."
```

Statistical explanation: Skewness is a measure of the asymmetry of a distribution:

- **Positive skewness (> 0):** right-skewed distribution with a longer right tail. The mean is greater than the median, since it is pulled upwards by the high values in the right tail.
- **Negative skewness (< 0):** left-skewed distribution with a longer left tail. The mean is smaller than the median, since it is pulled downwards by the low values in the left tail.
- **No skewness ($= 0$):** symmetric distribution, in which the mean and median are approximately equal.

Rules of thumb for interpreting skewness:

- $|\text{skewness}| < 0.5$: approximately symmetric
- $0.5 \leq |\text{skewness}| < 1$: moderately skewed
- $|\text{skewness}| \geq 1$: strongly skewed

Skewness is important for choosing appropriate statistical methods, since many procedures assume a symmetric distribution.

R explanation:

- The function `skewness()` from the `e1071` package calculates the skewness of a numeric vector.
- The functions `rbeta()` and `rnorm()` generate random numbers from the beta and normal distribution, respectively.
- The beta distribution with parameters `shape1 < shape2` is right-skewed, with `shape1 > shape2` it is left-skewed.
- The normal distribution is always symmetric (`skewness = 0`).

8 Measures of Association

Measures of association quantify the strength and direction of the relationship between two or more variables. They are fundamental for analysing dependencies and correlations in data.

8.1 Covariance

Covariance measures the linear association between two variables. It indicates whether the variables tend to rise and fall together (positive covariance) or move in opposite directions (negative covariance).

Mathematical formula for the sample covariance: $\text{cov}(X, Y) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$

```
# Inspect the first values of the variables
print("First values of 'sales':")
```

```
## [1] "First values of 'sales':"
```

```
head(sales)
```

```
## [1] 50 57 41 54 54 38
```

```
print("First values of 'adverts':")
```

```
## [1] "First values of 'adverts':"
```

```
head(adverts)
```

```
## [1] 2 5 1 3 4 1
```

```
# Manual calculation of the covariance
n_sales <- sales |> length()
cov_manual <- 1 / (n_sales - 1) * sum((sales - (sales |> mean())) *
                                     (adverts - (adverts |> mean())))

# Calculation using the built-in function
cov_builtin <- cov(sales, adverts)

print(paste("Covariance (manual):", round(cov_manual, 2)))
```

```
## [1] "Covariance (manual): 8.83"
```

```
print(paste("Covariance (built-in):", round(cov_builtin, 2)))
```

```
## [1] "Covariance (built-in): 8.83"
```

```
# Visualisation of the covariance
# Larger plotting window for better readability
par(mar = c(5, 4, 4, 2) + 0.5) # Increase the plot margins

plot(adverts, sales,
     main = "Scatterplot: Sales vs. Advertising",
     xlab = "Advertising spend",
     ylab = "Sales",
     pch = 19,
     col = "blue")

# Calculation of the means
mean_sales <- mean(sales)
mean_adverts <- mean(adverts)

# Draw the mean lines
abline(h = mean_sales, col = "red", lty = 2)
abline(v = mean_adverts, col = "red", lty = 2)

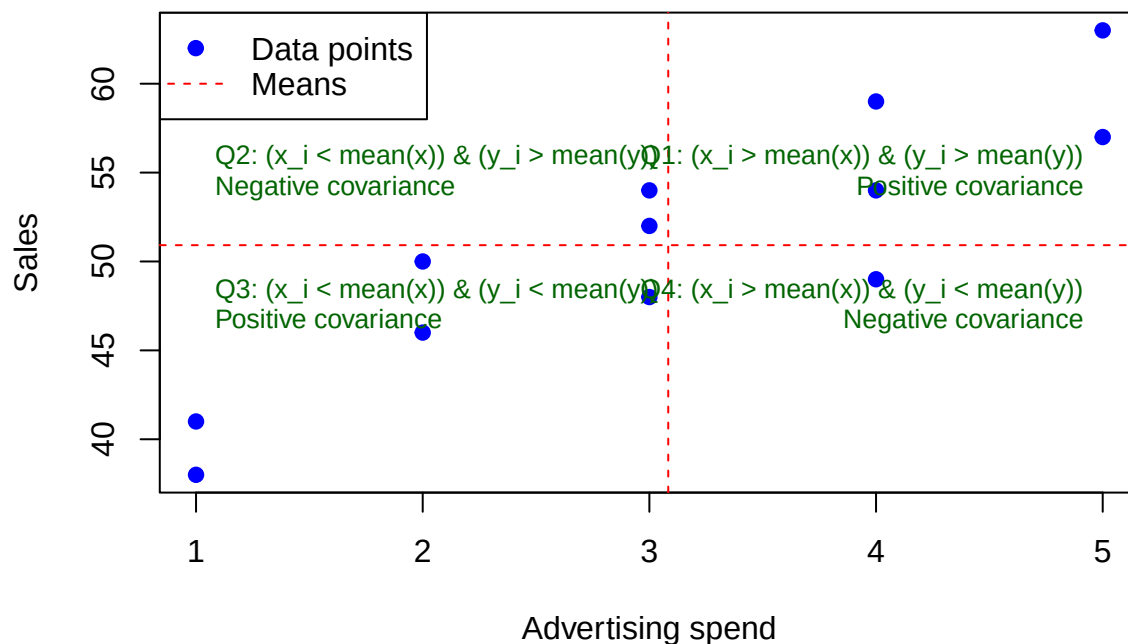
# Divide the plot into quadrants with better positioning and smaller text
text(min(adverts), mean_sales + 0.15*(max(sales)-min(sales)),
     "Q2: (x_i < mean(x)) & (y_i > mean(y))\nNegative covariance",
     pos = 4, col = "darkgreen", cex = 0.8)
text(max(adverts), mean_sales + 0.15*(max(sales)-min(sales)),
     "Q1: (x_i > mean(x)) & (y_i > mean(y))\nPositive covariance",
     pos = 2, col = "darkgreen", cex = 0.8)
text(min(adverts), mean_sales - 0.15*(max(sales)-min(sales)),
     "Q3: (x_i < mean(x)) & (y_i < mean(y))\nPositive covariance",
     pos = 4, col = "darkgreen", cex = 0.8)
text(max(adverts), mean_sales - 0.15*(max(sales)-min(sales)),
     "Q4: (x_i > mean(x)) & (y_i < mean(y))\nNegative covariance",
     pos = 2, col = "darkgreen", cex = 0.8)

# Reset the plot parameters after this plot
par(mar = c(5, 4, 4, 2)) # Restore default values

# Add a legend
```

```
legend("topleft",
      legend = c("Data points", "Means"),
      col = c("blue", "red"),
      pch = c(19, NA),
      lty = c(NA, 2))
```

Scatterplot: Sales vs. Advertising



Statistical explanation: Covariance measures how two variables vary together:

- **Positive covariance:** the variables tend to rise and fall together. When one variable is above its mean, the other tends to be above its mean as well.
- **Negative covariance:** the variables tend to vary in opposite directions. When one variable is above its mean, the other tends to be below its mean.
- **Covariance close to 0:** there is no linear association between the variables.

However, covariance has a drawback: its value depends on the units of the variables, which makes it difficult to compare across different pairs of variables. This problem is solved by the correlation coefficient.

R explanation:

- The function `cov()` calculates the covariance between two vectors.
- The function `abline()` with the parameters `h` and `v` adds horizontal and vertical lines to a plot.
- The parameter `lty` controls the line type (1 = solid, 2 = dashed, etc.).
- The function `text()` adds text at given coordinates.

8.2 Correlation Coefficient

The correlation coefficient (Pearson's r) standardises the covariance to the range $[-1, 1]$ by dividing it by the product of the standard deviations. This makes it independent of the units of the variables and allows direct comparisons across different pairs of variables.

$$\text{Mathematical formula: } r = \frac{\text{cov}(X,Y)}{s_X \cdot s_Y} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \sum_{i=1}^n (y_i - \bar{y})^2}}$$

```

# Manual calculation of the correlation coefficient
corr_manual <- cov(sales, adverts) / ((sales |> sd()) * (adverts |> sd()))

# Calculation using the built-in function
corr_builtin <- cor(sales, adverts)

print(paste("Correlation (manual):", round(corr_manual, 4)))

## [1] "Correlation (manual): 0.8884"

print(paste("Correlation (built-in):", round(corr_builtin, 4)))

## [1] "Correlation (built-in): 0.8884"

# Visualisation of the relationship
plot(adverts, sales,
     main = paste("Scatterplot: Sales vs. Advertising\nr =", round(corr_builtin, 2)),
     xlab = "Advertising spend",
     ylab = "Sales",
     pch = 19,
     col = "blue")

# Add a regression line
fit <- lm(sales ~ adverts)
abline(fit, col = "red", lwd = 2)

# Confidence interval for the regression line
newx <- seq(min(adverts), max(adverts), length.out = 100)
conf_interval <- predict(fit, newdata = data.frame(adverts = newx), interval =
  ↪ "confidence", level = 0.95)

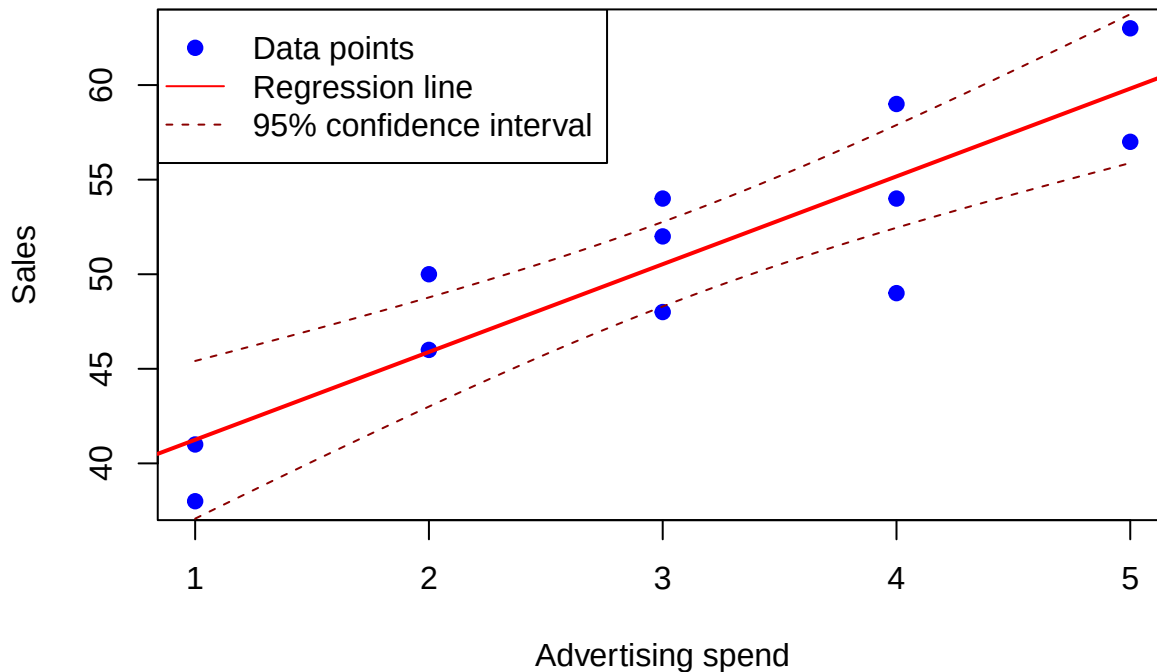
lines(newx, conf_interval[, "lwr"], col = "darkred", lty = 2)
lines(newx, conf_interval[, "upr"], col = "darkred", lty = 2)

# Add a legend
legend("topleft",
     legend = c("Data points", "Regression line", "95% confidence interval"),
     col = c("blue", "red", "darkred"),
     pch = c(19, NA, NA),
     lty = c(NA, 1, 2))

```

Scatterplot: Sales vs. Advertising

$r = 0.89$



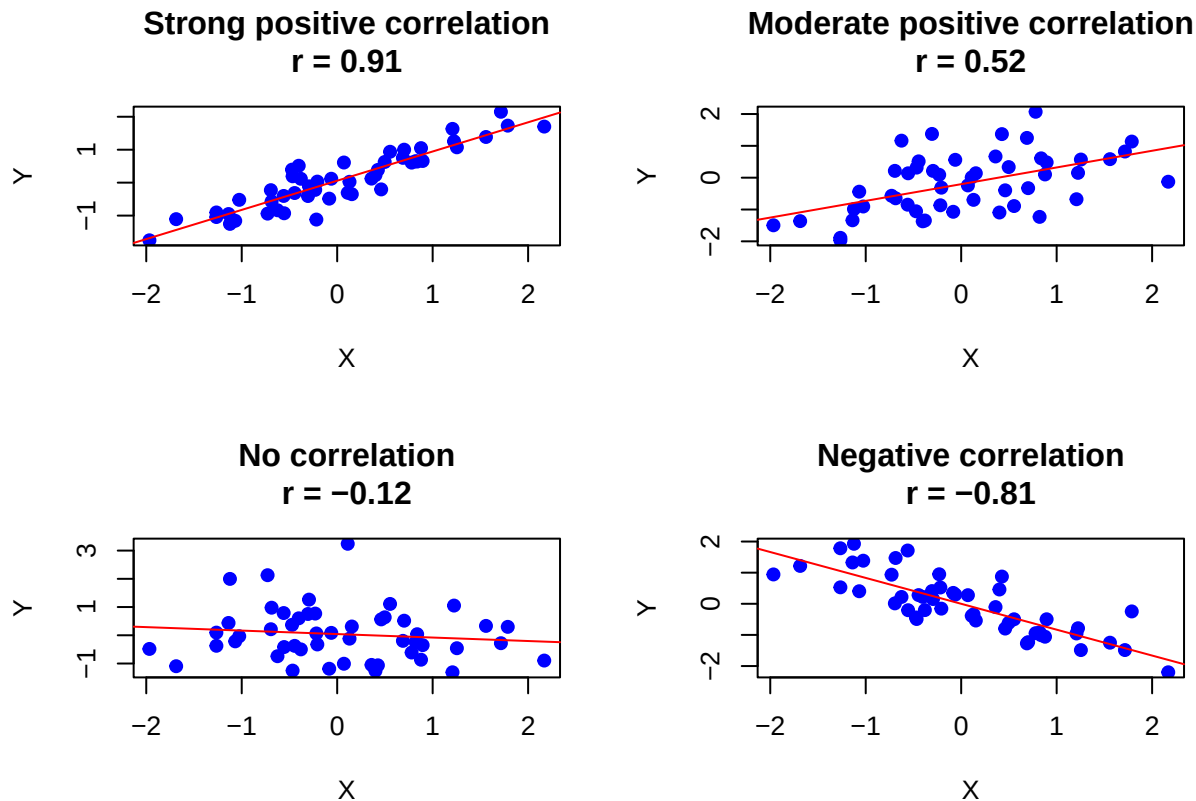
```
# Demonstration of different correlation strengths
par(mfrow = c(2, 2))

# Strong positive correlation (r ~ 0.9)
set.seed(123)
x1 <- rnorm(50)
y1 <- 0.9*x1 + rnorm(50, 0, 0.4)
plot(x1, y1, main = paste("Strong positive correlation\nr =", round(cor(x1, y1), 2)),
     xlab = "X", ylab = "Y", pch = 19, col = "blue")
abline(lm(y1 ~ x1), col = "red")

# Moderate positive correlation (r ~ 0.5)
y2 <- 0.5*x1 + rnorm(50, 0, 0.8)
plot(x1, y2, main = paste("Moderate positive correlation\nr =", round(cor(x1, y2), 2)),
     xlab = "X", ylab = "Y", pch = 19, col = "blue")
abline(lm(y2 ~ x1), col = "red")

# No correlation (r ~ 0)
y3 <- rnorm(50)
plot(x1, y3, main = paste("No correlation\nr =", round(cor(x1, y3), 2)),
     xlab = "X", ylab = "Y", pch = 19, col = "blue")
abline(lm(y3 ~ x1), col = "red")

# Negative correlation (r ~ -0.7)
y4 <- -0.7*x1 + rnorm(50, 0, 0.6)
plot(x1, y4, main = paste("Negative correlation\nr =", round(cor(x1, y4), 2)),
     xlab = "X", ylab = "Y", pch = 19, col = "blue")
abline(lm(y4 ~ x1), col = "red")
```

```
# Reset to a single plot
par(mfrow = c(1, 1))
```

Statistical explanation: The correlation coefficient (Pearson's r) measures the strength and direction of the linear relationship between two variables:

- $r = 1$: perfect positive linear relationship. All data points lie exactly on an increasing line.
- $r = -1$: perfect negative linear relationship. All data points lie exactly on a decreasing line.
- $r = 0$: no linear relationship. The variables are linearly independent.

Rules of thumb for interpreting the strength of correlation:

- $|r| < 0.3$: weak correlation
- $0.3 \leq |r| < 0.7$: moderate correlation
- $|r| \geq 0.7$: strong correlation

Important notes on interpretation:

1. Correlation does not imply causation. A strong relationship does not mean that one variable causes the other.
2. The correlation coefficient measures only linear relationships. Non-linear relationships can be strong despite $r \neq 0$.
3. The correlation coefficient is sensitive to outliers.
4. The coefficient of determination r^2 gives the proportion of variance in one variable that is explained by the other variable.

R explanation:

- The function `cor()` calculates the Pearson correlation coefficient between two vectors.
- With `method = "spearman"` or `method = "kendall"`, rank correlations can also be calculated.

- The function `predict()` with `interval = "confidence"` calculates confidence intervals for the predictions of a linear model.
- The function `set.seed()` sets the seed for the random number generator to obtain reproducible results.
- The function `rnorm()` generates normally distributed random numbers.

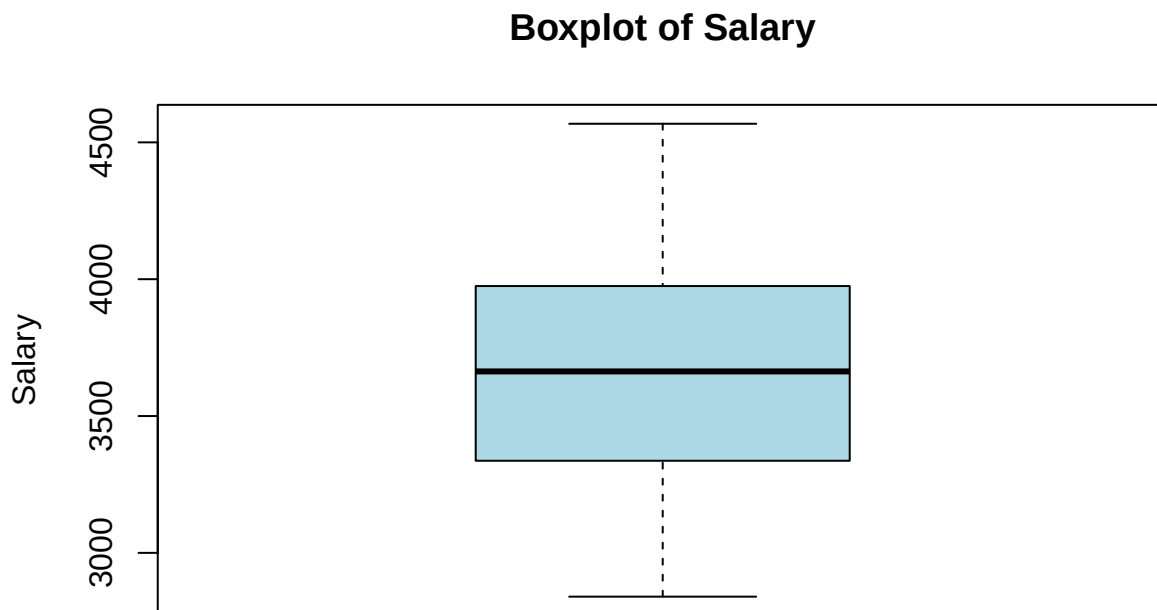
9 Visualisation

Visualisations are a powerful tool in descriptive statistics for detecting and communicating patterns, trends, and relationships in data. They complement numerical measures and help to grasp complex relationships intuitively.

9.1 Boxplots

Boxplots (also called box-and-whisker plots) show the distribution of the data via its quartiles. They are particularly useful for detecting outliers and for comparing groups.

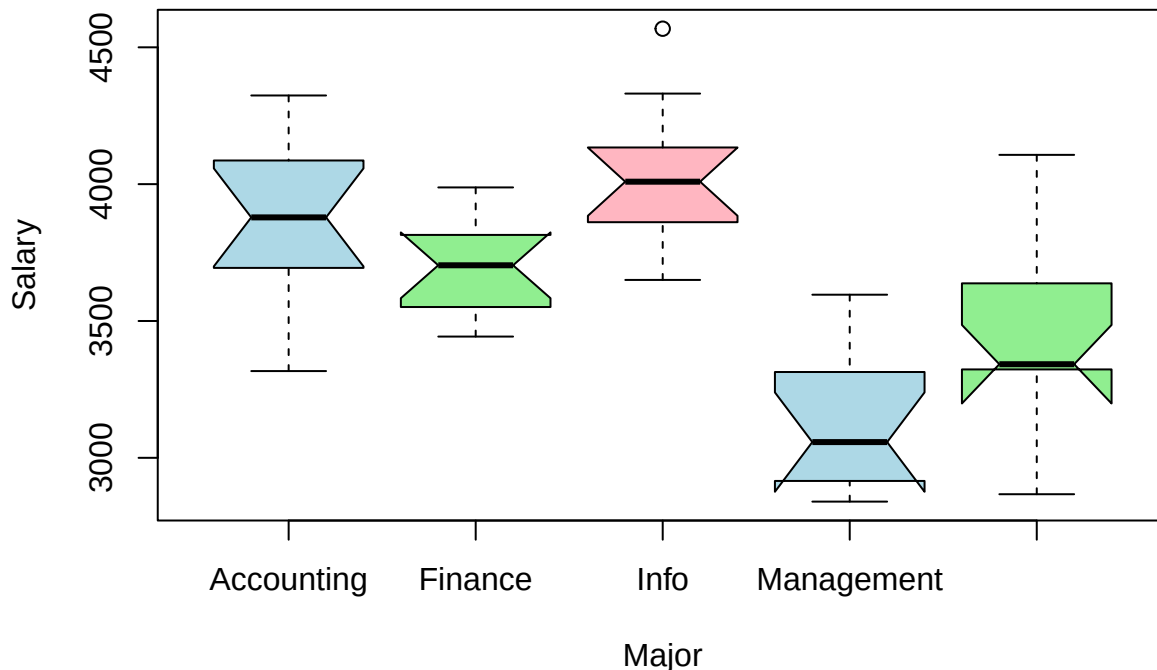
```
# Simple boxplot
boxplot(Salary,
        main = "Boxplot of Salary",
        ylab = "Salary",
        col = "lightblue",
        horizontal = FALSE, # Vertical orientation
        notch = FALSE)     # Without notch
```



```
# Grouped boxplot
boxplot(Salary ~ Major,
        data = Graduates,
        main = "Boxplot of Salary by Major",
        xlab = "Major",
        ylab = "Salary",
        col = c("lightblue", "lightgreen", "lightpink"),
        notch = TRUE) # With notch for median comparison
```

```
## Warning in (function (z, notch = FALSE, width = NULL, varwidth = FALSE, : some
## notches went outside hinges ('box'): maybe set notch=FALSE
```

Boxplot of Salary by Major



```
# Explanation of the boxplot elements with detailed labels
bp <- boxplot(Salary,
  main = "Anatomy of a Boxplot",
  ylab = "Salary",
  col = "lightblue",
  outline = TRUE, # Show outliers as points
  plot = TRUE) # Produce the plot

# Extract the boxplot statistics
stats <- bp$stats
outliers <- bp$out

# Label the boxplot elements
text(1.3, stats[5, 1], "Maximum (excl. outliers)", pos = 4, col = "darkblue")
text(1.3, stats[4, 1], "Upper quartile (Q3, 75%)", pos = 4, col = "darkblue")
text(1.3, stats[3, 1], "Median (Q2, 50%)", pos = 4, col = "darkblue")
text(1.3, stats[2, 1], "Lower quartile (Q1, 25%)", pos = 4, col = "darkblue")
text(1.3, stats[1, 1], "Minimum (excl. outliers)", pos = 4, col = "darkblue")

# Label the outliers, if any
if (length(outliers) > 0) {
  text(rep(1.1, length(outliers)), outliers, "Outlier", pos = 4, col = "red")
}

# Add lines for the IQR and whiskers
```

```

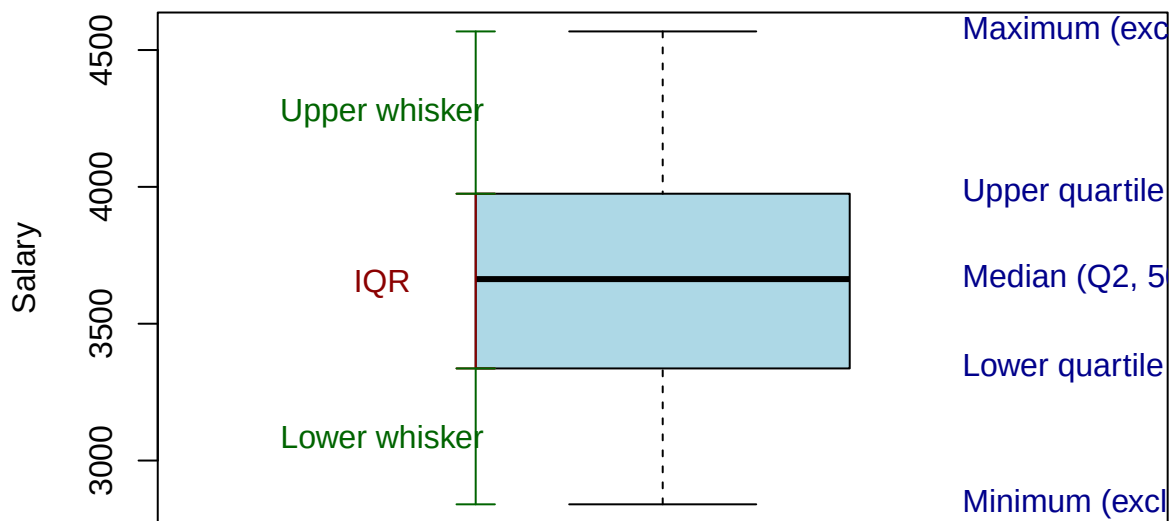
arrows(0.8, stats[2, 1], 0.8, stats[4, 1], code = 3, angle = 90, length = 0.1, col =
  ↪ "darkred")
text(0.7, (stats[2, 1] + stats[4, 1])/2, "IQR", col = "darkred")

arrows(0.8, stats[1, 1], 0.8, stats[2, 1], code = 3, angle = 90, length = 0.1, col =
  ↪ "darkgreen")
text(0.7, (stats[1, 1] + stats[2, 1])/2, "Lower whisker", col = "darkgreen")

arrows(0.8, stats[4, 1], 0.8, stats[5, 1], code = 3, angle = 90, length = 0.1, col =
  ↪ "darkgreen")
text(0.7, (stats[4, 1] + stats[5, 1])/2, "Upper whisker", col = "darkgreen")

```

Anatomy of a Boxplot



```

# Comparison of boxplots with other visualisations
par(mfrow = c(1, 3))

# Boxplot
boxplot(Salary,
        main = "Boxplot",
        ylab = "Salary",
        col = "lightblue")

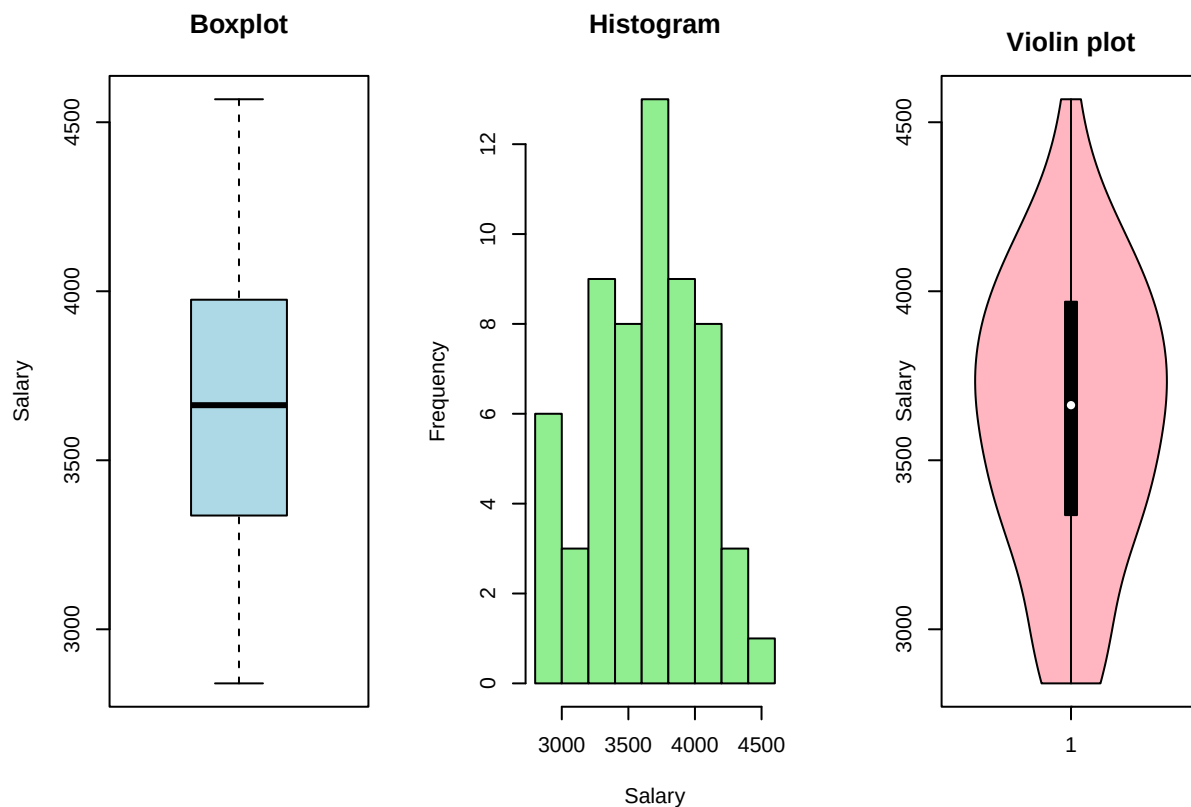
# Histogram
hist(Salary,
     main = "Histogram",
     xlab = "Salary",
     col = "lightgreen")

# Violin plot (combination of boxplot and density plot)
if (!requireNamespace("vioplot", quietly = TRUE)) {
  install.packages("vioplot")
}
library(vioplot)

```

```
## Loading required package: sm
## Package 'sm', version 2.2-6.0: type help(sm) for summary information
## Loading required package: zoo
##
## Attaching package: 'zoo'
## The following objects are masked from 'package:base':
##
##      as.Date, as.Date.numeric
```

```
vioplot(Salary,
        col = "lightpink",
        main = "Violin plot",
        ylab = "Salary")
```



```
# Reset to a single plot
par(mfrow = c(1, 1))
```

Statistical explanation: Boxplots provide a compact representation of the most important properties of a distribution:

1. **Box:** represents the interquartile range (IQR) from Q1 to Q3, which contains the middle 50% of the data.
2. **Line in the box:** shows the median (Q2).
3. **Whiskers:**
 - In R's default implementation: extend to the most extreme data point within $1.5 \times \text{IQR}$ from Q1

- or Q3.
 - Alternative definitions: to the minimum/maximum or to particular percentiles.
4. **Points outside the whiskers:** are considered outliers.
 5. **Notches:** when present, they indicate an approximate 95% confidence interval for the median. If the notches of two boxplots do not overlap, this is an indication of a significant difference between the medians.

Boxplots are particularly useful for:

- Comparing several groups
- Identifying outliers
- Assessing the symmetry of a distribution
- Detecting differences in spread

R explanation:

- The function `boxplot()` creates a boxplot.
- The parameter `notch = TRUE` adds notches that represent an approximate 95% confidence interval for the median.
- The parameter `horizontal = TRUE` creates a horizontal rather than a vertical boxplot.
- The function `boxplot()` returns a list of statistics that can be used for further annotations.
- The function `arrows()` adds arrows to the plot, used here to label the boxplot elements.
- The package `vioplot` provides an alternative to the boxplot that additionally shows the density of the data.

Tip: Boxplots are particularly useful for detecting outliers and comparing groups. The box shows the interquartile range (IQR), the line in the middle is the median, and the “whiskers” show the range of the data without outliers.